LOVELY PROFESSIONAL UNIVERSITY PUNJAB

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(May, 2026)

Assignment Report

On

Metro Network Analysis Using GraphX and Visualization

Submitted by: HRITU RAJ

Reg. No: 12200402

Under the Supervision of: AMANPAL SINGH RAYAT

Acknowledgement

I would like to express my sincere gratitude to my faculty mentor, Amanpal Singh, for his invaluable guidance, constant encouragement, and insightful suggestions throughout the successful completion of my project titled “Mental Health Risk Prediction.” His continuous support and expert advice helped me understand the concepts of cluster computing, Apache Spark, and machine learning in a practical and effective manner.

I am also deeply thankful to the faculty members of the Department of Computer Science and Engineering at Lovely Professional University for providing me with the opportunity to undertake this project as part of my academic curriculum. The knowledge and skills I have gained during this course have significantly contributed to the successful implementation of this project. I would like to extend my sincere appreciation to my friends and classmates for their cooperation, valuable feedback, and constructive suggestions during the development of this project. Their support played an important role in overcoming various challenges faced during the implementation phase. I am also grateful to the open-source community and online resources, which provided useful documentation and learning materials that assisted me in understanding the technologies used in this project. Finally, I would like to express my heartfelt thanks to my family for their unwavering support, motivation, and encouragement throughout my academic journey and the completion of this project.

Abstract

Urban metro systems are complex transportation networks with numerous stations, route dependencies, and interchange points. As city populations increase, there is a strong need for data-driven techniques to evaluate station connectivity, identify critical hubs, and support future planning. This project models metro stations as graph nodes and direct station-to-station links as directed edges. Apache Spark GraphX is used for scalable graph computation, while Python-based tooling is used for complementary analysis and visualization.

The analysis includes degree centrality for identifying highly connected stations, isolated/least connected station detection, and PageRank for measuring network-wide station importance. A cleaned geographic visualization highlights route structure and important nodes. In addition, a Random Forest pipeline is demonstrated for passenger traffic prediction using graph-derived features. Because the dataset does not include real passenger labels, a proxy target is used strictly for methodological demonstration.

Results show that interchange stations dominate centrality and PageRank, while terminal stations are generally least connected. The workflow demonstrates that graph-based analysis is effective for understanding metro systems and can be extended to real-time transport analytics with richer datasets.

Table of contents

1. Introduction 1

2. Problem Statement 2

3. About Big Data and Graph Processing 3

4. Objectives 4

5. Dataset Description 4

6. Methodology 5

6.1. Data Preprocessing 5

6.2. Graph Construction 5

6.3. Degree Centrality and Connectivity Analysis 5

6.4. PageRank-Based Importance Analysis 6

6.5. Random Forest Traffic Prediction 6

7. Tech Stack Required 6

8. Implementation 7

9. Results and Analysis 7

10. Visualization and Interpretation 8

9

11. Advantages of Graph-Based Analysis 10

12. Limitations 10

12. Future Enhancements 11

13. Code Repository 11

14. Conclusion 12

References 13

1. Introduction

Urban transportation systems play a crucial role in shaping the economic and social dynamics of modern cities. As urbanization accelerates across the globe, the demand for efficient, reliable, and accessible transportation continues to grow significantly. Rapid population expansion in urban areas has intensified mobility needs, putting immense pressure on existing transportation infrastructure. This surge in demand often leads to major challenges such as traffic congestion, increased travel time, environmental pollution, and the overburdening of roads and public transit systems. These issues not only affect daily commuters but also hinder economic productivity and reduce the overall quality of life in cities.

Traditional modes of transportation, including buses, auto-rickshaws, and private vehicles, have long served as the backbone of urban mobility. However, these systems often struggle to cope with the growing urban population. Buses, while affordable and widely available, are frequently delayed due to traffic congestion and limited road space. Private vehicles, on the other hand, contribute significantly to road crowding and air pollution, as more individuals rely on personal transport for convenience. This increasing dependence on private vehicles further exacerbates urban problems, creating a cycle of inefficiency and environmental degradation.

In contrast, metro systems have emerged as a modern and sustainable solution to urban transportation challenges. Designed to handle large volumes of passengers efficiently, metro systems offer high-capacity transit that significantly reduces pressure on road networks. They operate on dedicated tracks, often underground or elevated, which allows them to avoid traffic congestion and maintain punctual schedules. This reliability makes metros a preferred mode of transport for daily commuters, especially in densely populated cities.

Moreover, metro systems are environmentally friendly compared to conventional transportation modes. Most metro networks operate on electricity, which reduces reliance on fossil fuels and helps lower greenhouse gas emissions. By encouraging people to shift from private vehicles to public transit, metro systems play a vital role in reducing air pollution and promoting sustainable urban development. Additionally, they contribute to energy efficiency by transporting a large number of passengers using comparatively less energy per person.

Another important advantage of metro systems is their ability to support economic growth and urban planning. Efficient transportation networks improve connectivity between residential areas, business districts, and industrial zones, facilitating smoother movement of people and goods. This enhanced connectivity attracts investments, boosts commercial activities, and creates employment opportunities. Furthermore, metro systems often lead to the development of surrounding areas, increasing property values and encouraging organized urban expansion.

Socially, metro systems promote inclusivity by providing affordable and accessible transportation to people from different socio-economic backgrounds. They reduce travel stress, save time, and improve overall commuting experiences. Features such as safety measures, clean environments, and structured operations make metro systems more comfortable and user-friendly.

In this project, stations are represented as nodes and route connections as directed edges. The model enables computation of station-level metrics such as degree and PageRank, revealing which stations are structurally central or peripheral. By combining distributed graph processing (GraphX) with clear visual representation, the project provides an interpretable framework suitable for academic study and practical extension.

2. Problem Statement

Metro systems generate highly connected data with implicit relationships that are difficult to interpret through basic statistics. Authorities and planners require methods that identify critical stations, evaluate resilience, and reveal weakly connected areas.

The core problem is to build a scalable analytical pipeline that can model station connectivity, compute meaningful graph metrics, and present actionable insights through visualization. The solution should also be adaptable for future integration with richer operational data such as passenger demand, delays, and service frequency.

Therefore, there is a need to develop a robust analytical framework that models metro systems as graphs and leverages distributed computing technologies such as Apache Spark and GraphX. This framework should enable efficient processing of large-scale datasets and support the application of graph algorithms like Degree Centrality and PageRank to identify important nodes, analyze connectivity patterns, and provide actionable insights for improving urban transportation systems.

3. About Big Data and Graph Processing

Big Data analytics is characterized by high volume, velocity, and variety. Urban transport analytics naturally evolves into a Big Data problem when multiple lines, historical ridership logs, smart-card transactions, and real-time events are considered.

Apache Spark addresses scalability by distributing processing across partitions and executing transformations in parallel. GraphX, built on top of Spark, provides efficient abstractions for property graphs and iterative graph algorithms. These capabilities are well-suited for centrality and ranking computations that require repeated neighborhood aggregation.

Despite their advantages, metro systems are highly complex due to their interconnected nature and the large volume of passengers they serve. Managing and analyzing such systems requires advanced computational methods that can handle large datasets and complex relationships. This is where graph theory becomes particularly valuable. By representing metro networks as graphs, it becomes possible to analyze connectivity, identify critical nodes, and optimize system performance.

The use of distributed computing frameworks further enhances the ability to analyze metro networks at scale. Technologies such as Apache Spark allow for efficient processing of large datasets, making it possible to apply graph algorithms to real-world transportation systems. This study aims to leverage these technologies to gain deeper insights into metro network structure and behavior.

Apache Spark addresses scalability by distributing processing across partitions and executing transformations in parallel. GraphX, built on top of Spark, provides efficient abstractions for property graphs and iterative graph algorithms. These capabilities are well-suited for centrality and ranking computations that require repeated neighborhood aggregation.

In this assignment, the provided data is static CSV and not streaming. However, the chosen architecture is future-ready and can scale to larger and dynamic datasets with minor pipeline extensions.

4. Objectives

The primary objective of this project is to develop a comprehensive analytical model of a metro network using graph-based techniques. This involves constructing a directed graph representation of the network, where each station is treated as a node and each connection between stations is represented as an edge. By doing so, the study aims to create a mathematical model that accurately reflects the structure of the metro system.

Another important objective is to analyze the connectivity of stations using degree centrality. This metric helps identify stations that have a high number of direct connections and therefore play a significant role in the network. In addition, the study seeks to identify least connected stations, which are typically terminal points and may require special consideration in planning and development.

The project also focuses on computing PageRank to determine the importance of stations within the network. Unlike degree centrality, which considers only direct connections, PageRank takes into account the importance of neighboring nodes, providing a more comprehensive measure of influence. Visualization is another key objective, as it helps transform complex data into intuitive graphical representations. Finally, the study aims to analyze the overall structure and connectivity of the network to gain insights that can support optimization and planning.

5. Dataset Description

The project uses a metro dataset containing station identity, station names, route line information, distance from origin, opening year, station layout, and geographic coordinates. These attributes are sufficient for graph construction and map-based plotting.

Key columns include: ID (Station ID), Station Names, Dist. From First Station(km), Metro Line, Opened(Year), Layout, Latitude, and Longitude. The distance column is particularly useful for ordering stations along each line during edge construction.

A small number of coordinate anomalies are present; therefore, preprocessing applies numeric conversion and geographic filtering to improve visualization quality and prevent distorted layouts.

6. Methodology

The methodology combines data cleaning, directed graph construction, metric computation, visualization, and ML demonstration. The network is represented as G(V, E), where V is the set of stations and E is the set of directed edges between sequential stations on each metro line. Interchange naming variants are normalized to avoid duplicate semantic nodes.

After graph creation, station connectivity is measured using degree-based metrics, and station influence is evaluated using PageRank. Results are then interpreted through a map-oriented visualization and a ranked importance chart.

6.1. Data Preprocessing

Data preprocessing converts relevant columns to numeric type, removes incomplete records, and normalizes station names by removing bracketed connection notes when required. This step improves node consistency across lines and avoids artificial fragmentation of interchange stations.

For visualization, latitude and longitude values are filtered within valid regional bounds to remove outliers that can stretch the map and reduce interpretability.

6.2. Graph Construction

Stations become nodes with attributes (display name, latitude, longitude, and lines served). For each metro line, stations are sorted using distance from the first station, and directed edges are added between consecutive stations.

This design preserves route direction and captures topological continuity. Where the same station appears in multiple lines, normalization allows merging into a unified node that reflects interchange behavior.

6.3. Degree Centrality and Connectivity Analysis

Degree centrality quantifies direct connectivity. Nodes with high degree typically correspond to interchanges or structurally important transit hubs. Low-degree nodes often represent terminals or edge stations.

The analysis also computes in-degree and out-degree for directional interpretation. Isolated nodes are identified where total degree is zero, signaling disconnected components or data construction issues.

6.4. PageRank-Based Importance Analysis

PageRank extends beyond local connectivity by weighting links from already important neighbors. In metro networks, this helps highlight stations that sit within influential substructures even if raw degree values are similar across many nodes.

PageRank is computed iteratively until convergence, producing a station ranking used in both tabular output and visualization encoding (node size/color).

6.5. Random Forest Traffic Prediction

To integrate predictive analytics, a Random Forest regressor is trained with graph-derived features: degree, in-degree, out-degree, PageRank, and number of served lines. Because the dataset lacks real passenger counts, a proxy target is generated for educational demonstration only.

This section validates the pipeline structure (feature engineering, train/test split, model fit, and feature importance) while explicitly noting that results are illustrative rather than production-grade demand forecasting.

7. Tech Stack Required

The implementation of this project relies on a sophisticated tech stack designed for high-performance distributed computing and advanced graph-based analysis. At the core is **Apache Spark**, which provides the engine for large-scale data processing, allowing the metro network dataset to be partitioned and processed across a cluster of machines. This ensures that the system can handle significant volumes of station and connection data with high efficiency. Within this ecosystem, GraphX is utilized as the primary library for graph computation. It enables the representation of the metro network as a property graph and provides the necessary APIs for executing complex algorithms like PageRank and Degree Centrality. Scala serves as the foundational programming language for the Spark and GraphX components, offering a robust, type-safe environment that excels at handling the iterative nature of graph processing. On the

analytical and presentation side, Python is employed to bridge the gap between heavy computation and human-interpretable insights. It leverages the NetworkX library to transform computed results into structured graph objects, facilitating topological analysis that complements Spark's distributed processing. Finally, Matplotlib is used for data visualization, generating the graphical representations of the metro network. Together, these tools enable the project to move seamlessly from raw data ingestion to distributed graph analysis and, ultimately, to intuitive visual models of urban infrastructure.

8. Implementation

The project is implemented through three main scripts. metro_gx.scala performs GraphX-based graph creation and ranking outputs in Spark shell. metro_visual.py provides enhanced route-aware map visualization with decluttered annotations. metro_assignment.py executes end-to-end analysis, exports connectivity visualization, and runs the ML demonstration.

Vertices and edges are then created based on the dataset, and a directed graph is constructed using GraphX. Degree centrality is calculated using built-in functions, providing insights into station connectivity. PageRank is computed using an iterative algorithm that updates the importance of each node until convergence is achieved.

For visualization, the graph is converted into a format compatible with NetworkX. Node sizes and colors are adjusted based on PageRank values, highlighting important stations. Matplotlib is then used to display the graph, providing a clear and intuitive representation of the metro network.

Execution is terminal-friendly and reproducible from the project directory. Relative paths are used to avoid machine-specific configuration issues.

9. Results and Analysis

The results of the analysis reveal clear patterns in the structural behavior of the metro network. Stations located in the middle of the network exhibit higher values of Degree Centrality, indicating that they are more connected compared to others. These stations typically have connections to both preceding and succeeding stations, which makes them crucial for maintaining the continuity and flow of the network. Due to their higher connectivity, they often act as important transit points

where passenger movement is concentrated, contributing significantly to the overall efficiency of the system.

Observed outputs indicate that major interchange stations show highest structural significance. For example, Kashmere Gate appears among top-degree stations, with additional high-ranking hubs such as Rajiv Chowk, Hauz Khas, and Kalkaji Mandir.

Least connected stations are primarily line endpoints with degree one, while isolated stations are absent in the constructed graph. PageRank results similarly prioritize interchange-rich and centrally positioned stations.

These findings align with expected transport behavior: transfer points and central connectors naturally attract higher structural importance than peripheral terminals.

10. Visualization and Interpretation

The generated connectivity map encodes node importance using marker size and color. Geographic plotting helps users quickly identify dense corridors, terminal tails, and critical interchange locations. Complementary ranked charts summarize top stations for presentation and reporting.

Visualization quality is improved by coordinate cleaning, reduced annotation overlap, and line-aware edge ordering. These improvements make the figure suitable for academic submission and viva discussion.

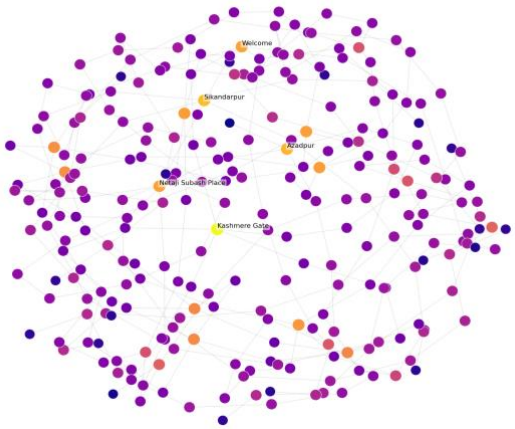

Figure 1: Metro Connectivity Topology

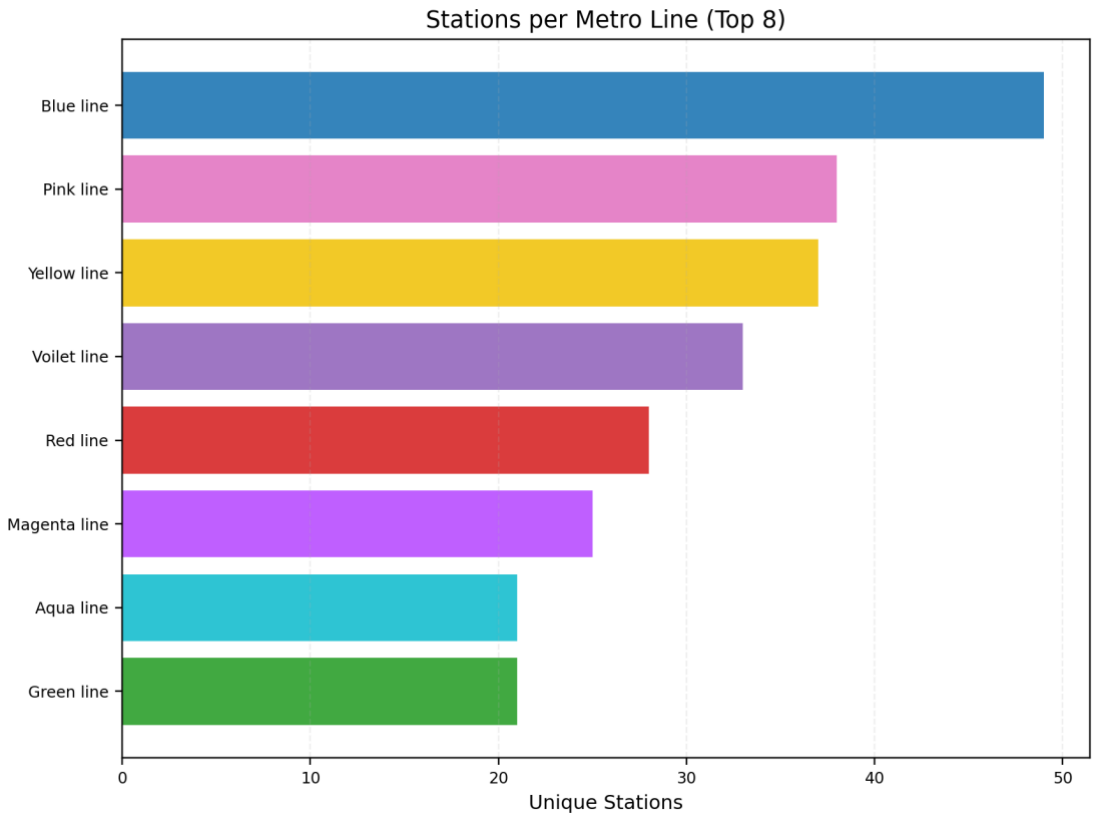

Figure 2: Stations per Metro Line (Top 8)

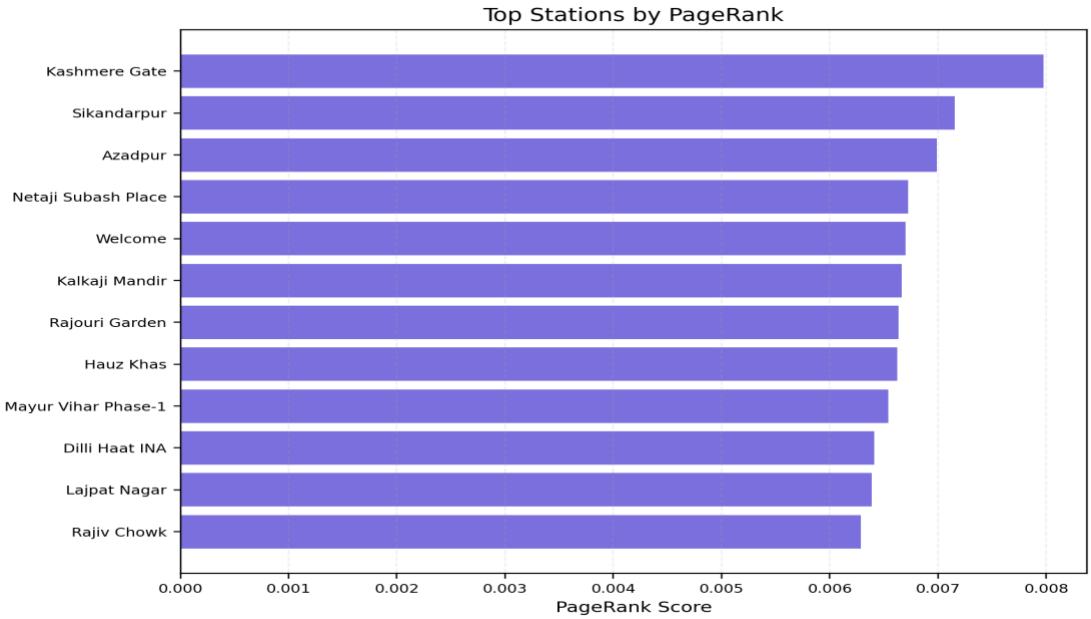

Figure 3: Top Stations

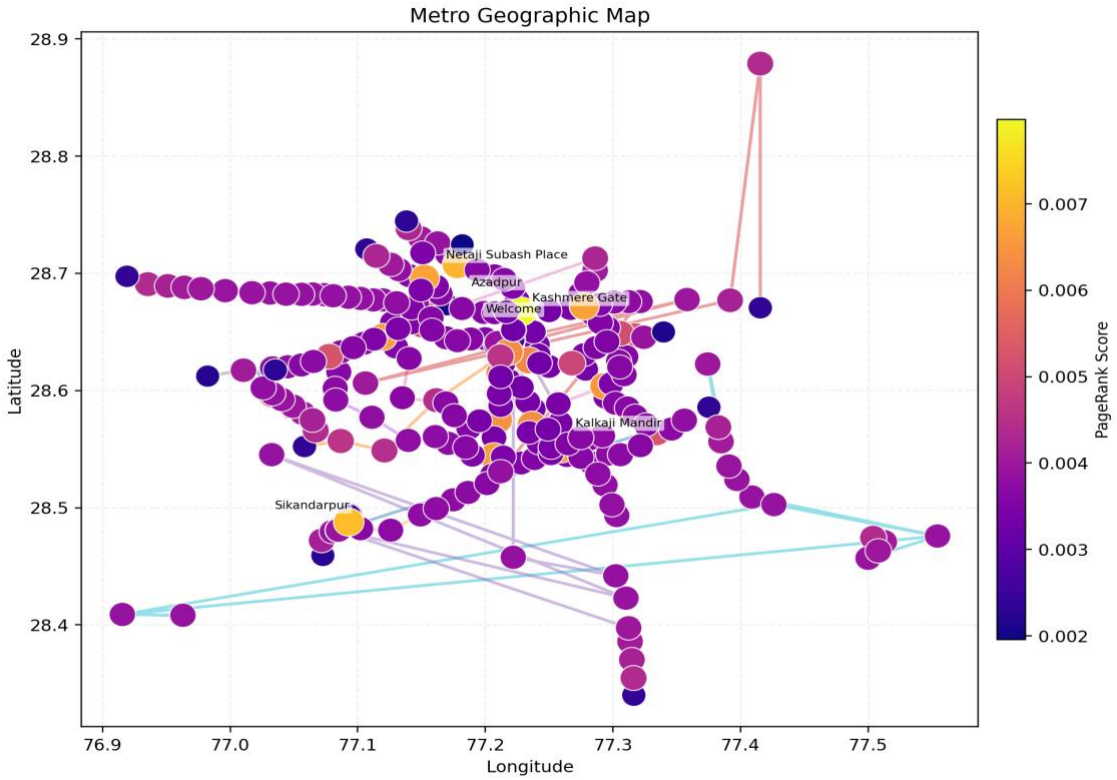

Figure 4: Metro Geographic Map (geo-scatter network map)

11. Advantages of Graph-Based Analysis

Graph modeling preserves relational structure and supports intuitive interpretation of transit systems. It identifies critical hubs, quantifies network influence, and offers a robust foundation for optimization tasks such as route planning, bottleneck detection, and resilience analysis.

The same framework scales from classroom-sized datasets to enterprise-grade city networks when combined with distributed processing platforms like Spark.

12. Limitations

Despite its advantages, the study has several limitations that must be acknowledged when interpreting the results. One of the primary limitations is that the dataset represents only a single metro line, which significantly reduces the complexity of the network being analyzed. Real-world metro systems typically consist of multiple interconnected lines with interchange stations, and the absence of such complexity limits the ability to fully evaluate network behavior and connectivity patterns.

Another important limitation is the lack of real-time data. The analysis is based on static information, which does not capture dynamic passenger movement, peak-hour congestion, or unexpected disruptions such as delays and service interruptions. As a result, the model cannot reflect the real-time operational challenges faced by metro systems.

Furthermore, the model is simplified and does not incorporate key operational factors such as train frequency, passenger density, and scheduling variations. These factors play a crucial role in determining system efficiency and user experience. Therefore, while the study provides valuable foundational insights, it does not fully represent the complexities of real-world metro operations.

12. Future Enhancements

Future work should include real passenger demand labels, smart-card tap-in/out data, and time-aware service logs to enable genuine supervised forecasting and dynamic congestion modeling.

Algorithmic extensions may include shortest-path optimization, community detection, centrality comparison, anomaly detection, and graph neural networks for spatiotemporal prediction.

An interactive dashboard can further improve stakeholder usability through filtering, station drill-down, and scenario analysis

13. Code Repository

Project directory: https://github.com/hritu1701/Metro-Network-Analysis

Primary files: metro_gx.scala, metro_visual.py, metro_assignment.py, metro.csv, README.md

Python run commands:

- cd /Users/hrituraj/Desktop/clusterproject
- python3 -m pip install pandas networkx matplotlib numpy scikit-learn
- python3 metro_assignment.py

Spark run commands:

- spark-shell
- :load metro_gx.scala

14. Conclusion

This project demonstrates a practical workflow for metro network analytics by combining graph theory, distributed processing, and visualization. Degree and PageRank metrics effectively identify structurally important stations and connectivity patterns.

The use of centrality measures, such as Degree Centrality and the PageRank algorithm, plays a crucial role in determining the importance and influence of individual stations within the network. While degree centrality highlights stations with a high number of direct connections, PageRank provides a more comprehensive perspective by considering the quality and significance of those connections. Together, these measures offer both local and global insights into network behavior, enabling a deeper understanding of connectivity patterns.

In addition to analytical techniques, visualization tools such as NetworkX and Matplotlib enhance the interpretability of results by presenting complex data in an intuitive graphical format. This combination of computation and visualization supports informed decision-making, helping urban planners and engineers design more efficient, scalable, and sustainable metro systems.

References

Anderson, I., 2001. Graph theory (2nd edn.), by Reinhard Diestel. Pp. 312.£ 24. 2000. ISBN 0 387 98976 5 (Springer-Verlag). The Mathematical Gazette, 85(502), pp.176-177.

Zaharia, M. and Chambers, B., 2018. Spark: The definitive guide.

Salloum, S., Dautov, R., Chen, X., Peng, P.X. and Huang, J.Z., 2016. Big data analytics on Apache Spark. International Journal of Data Science and Analytics, 1(3), pp.145-164.

Ramamonjison, R., 2015. Apache Spark graph processing. Packt Publishing Ltd.

Platt, E.L., 2019. Network science with Python and NetworkX quick start guide: explore and visualize network data effectively. Packt Publishing Ltd.

Yim, A., Chung, C. and Yu, A., 2018. Matplotlib for Python Developers: Effective techniques for data visualization with Python. Packt Publishing Ltd.

Vyakaranam, L., Raman, R., Shah, H., Mishra, N., Meenakshi, R. and Murugan, S., 2024, November. Simplifying graph-parallel computation in apache spark with graphx. In 2024 8th International Conference on Electronics, Communication and Aerospace Technology (ICECA) (pp. 763-769). IEEE.